Laboratório Prolog · Aula 13

Pietr · RA 24051001-2 · Turma B
Gerado em 20/10/2026, 20:34:47 · tempo registrado no cronômetro: 60:54

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

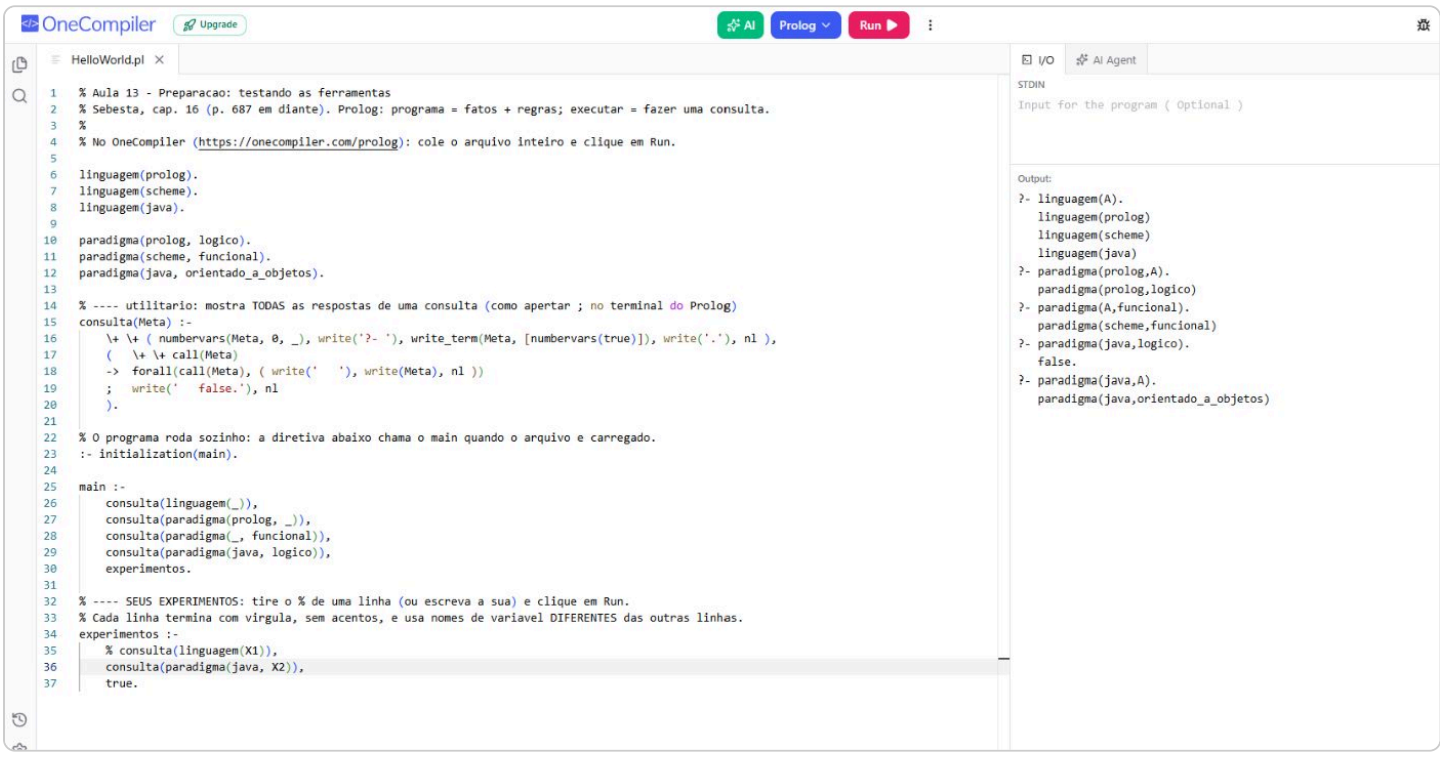
paradigma(L, funcional) vai responder L = scheme, porque existe o fato paradigma(scheme, funcional).
paradigma(java, logico) vai responder false, porque não existe esse fato no programa (Java está como orientado_a_objetos)

Saída do ola.pl

```
1- ?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.

2-?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```





0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem uma variável (P), então pergunta “qual é o paradigma do Prolog?”. O Prolog procura um valor para P e responde P = logico. Já paradigma(prolog, logico) não tem variável, então pergunta “é verdade que o Prolog é lógico?”. A resposta só pode ser true ou false, e aqui é true, porque esse fato está no programa.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não sabe isso, isto é, não consegue provar. O Prolog só conhece os fatos escritos no arquivo, e lá só existe paradigma(java, orientado_a_objetos). Pela hipótese do mundo fechado, tudo o que não está na base nem pode ser deduzido dela é considerado falso. O false é sobre o que o programa sabe, não sobre a realidade.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas: lucia, marta, tereza, joana, bia, ana e dora, na ordem em que aparecem no arquivo. (mae(M, F), mulher(F)) terá 4 pares, que são as mães com filhas mulheres: lucia e tereza, marta e joana, tereza e bia, joana e ana

Saída do familia.pl

```
1- --- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)

--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
```

```
    mulher(tereza)
    mulher(joana)
    mulher(bia)
    mulher(ana)
    mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mae(A,B),homem(B).
   mae(lucia,vern),homem(vern)
   mae(marta,paulo),homem(paulo)
   mae(joana,jake),homem(jake)
   mae(dora,leo),homem(leo)
?- homem(A),mulher(A).
   false.
?- mae(joana,jake),mae(joana,ana).
   mae(joana,jake),mae(joana,ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta pergunta “quais mães têm filhas mulheres?”, ou seja, os pares (mãe, filha) em que a filha é mulher. Ela teve 4 respostas: lucia e tereza, marta e joana, tereza e bia, joana e ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: tudo o que não está na base e não pode ser provado é considerado falso. Como não existe nenhum fato sobre maria, o Prolog responde false. Isso pode ser um problema em situações reais porque as bases costumam ser incompletas, e uma informação que apenas não foi cadastrada vira um “não”. Por exemplo, um paciente sem alergia registrada seria tratado como não alérgico.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas apareceram na ordem lucia, marta, tereza, joana, bia, ana e dora. Essa é a ordem em que os fatos mulher estão escritos no arquivo, porque o Prolog percorre a base de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) vai responder paulo, porque paulo é irmão de joana, que é mãe de jake. primo(leo, X) vai responder jake e ana, porque o pai de leo (paulo) é irmão da mãe deles (joana)

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Vão falhar: f(X, X) = f(a, b), porque X não pode ser a e b ao mesmo tempo; [A|B] = [], porque a lista vazia não tem cabeça; e 2 + 3 = 5, porque = não calcula, e a estrutura 2+3 é diferente do número 5. Só f(X, X) = f(a, Y) unifica, com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.

--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
```

```
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
?- irmao(A,B).
   irmao(vern,tereza)
   irmao(tereza,vern)
   irmao(joana,paulo)
   irmao(paulo,joana)
   irmao(jake,ana)
   irmao(ana,jake)
?- tio(A,B).
   tio(vern,bia)
   tio(paulo,jake)
   tio(paulo,ana)
?- ancestral(A,leo).
   ancestral(dora,leo)
   ancestral(paulo,leo)
   ancestral(lucia,leo)
   ancestral(marta,leo)
   ancestral(osvaldo,leo)
   ancestral(vern,leo)
?- primo(A,B).
   primo(bia,joana)
   primo(bia,paulo)
   primo(jake,leo)
   primo(ana,leo)
   primo(joana,bia)
   primo(paulo,bia)
   primo(leo,jake)
   primo(leo,ana)
[vern,tereza,joana,paulo,jake,ana,leo,bia]


--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
```

```
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao

--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica
f(A,B)  =  f(1,2)    unifica   A = 1   B = 2
g(A,b)  =  g(a,A)    NAO unifica
[A,B|C]  =  [1,2,3,4,5]    unifica   A = 1   B = 2   C = [3,4,5]
2+3  =  5    NAO unifica
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque a recursão precisa de um caso que para e de um caso que avança. O caso base é ancestral(X, Y) :- progenitor(X, Y): pai ou mãe é ancestral. O caso recursivo é ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y): X é ancestral de Y se é pai ou mãe de alguém (Z) que é ancestral de Y. Sem o caso base a recursão não para; sem o caso recursivo só se encontra uma geração.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X == Y, a consulta irmao(jake, _) passou a mostrar irmao(jake, jake) além de irmao(jake, ana). Isso acontece porque jake tem a mesma mãe e o mesmo pai que ele mesmo, então a regra o considera irmão de si próprio. A meta X == Y é necessária para garantir que X e Y sejam pessoas diferentes.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X, X) = f(a, b), o primeiro argumento faz X = a, e o segundo exigiria X = b. Como a é diferente de b, não unifica. Em f(X, X) = f(a, Y), o primeiro argumento faz X = a, e o segundo unifica X com Y, então Y também vale a. Ficam X = a e Y = a

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Não com uma única função. Em Java ou Scheme, uma função tem entradas e uma saída fixas, então seria preciso escrever duas: uma para os avós de um neto e outra para os netos de um avô. No Prolog, avo é uma relação. A unificação liga a variável em qualquer argumento, e assim a mesma regra funciona nas duas direções: avo(X, jake) e avo(osvaldo, X).

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1, 2, 3]) vai responder todas as formas de dividir [1, 2, 3] em duas listas: [] e [1,2,3]; [1] e [2,3]; [1,2] e [3]; [1,2,3] e []. São 4 respostas.

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])

?- concatena(A,[b,c],[a,b,c]).
   concatena([a],[b,c],[a,b,c])
?- inverte([a,b,c,d],A).
   inverte([a,b,c,d],[d,c,b,a])
?- tamanho([x,y],A).
   tamanho([x,y],2)
?- membro(A,[]).
   false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([], 0): a lista vazia tem tamanho 0. O passo recursivo é tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1: calcula o tamanho da cauda e soma 1. Em tamanho([a, b, c], N), cada chamada deixa pendente o “somar 1”: tamanho([b,c]), depois tamanho([c]), depois tamanho([]) = 0. Na volta da recursão, os valores ficam 1, 2 e, por fim, N = 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque existem 4 maneiras de dividir [1, 2, 3] em duas partes, e o backtracking encontra todas. O anexar do Scheme é uma função: ele só junta duas listas que você já deu e devolve o resultado. Ele não consegue fazer o contrário, isto é, descobrir quais listas, juntas, formam (1 2 3).

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Foi o backtracking. A primeira cláusula de membro dá a cabeça da lista (a). Quando o Prolog procura outra resposta, ele volta e tenta a segunda cláusula com a cauda, encontrando b e depois c, sem nenhum laço escrito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 mostra X = 2+3, a estrutura, sem calcular. X is 2 + 3 mostra X = 5, porque calcula. fatorial(N, 120) dá erro de instanciação, porque o Prolog tenta fazer N > 0 com N ainda sem valor.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error

?- A is 5*4+2.
   22 is 5*4+2
?- A=5*4+2.
   5*4+2=5*4+2
?- fatorial(10,A).
   fatorial(10,3628800)
?- fib(20,A).
   fib(20,6765)
?- fatorial(A,120).
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

O = unifica sem calcular: X = 2 + 3 faz X valer a estrutura 2+3, e 2 + 3 = 5 dá false. O is calcula o lado direito e unifica o resultado com o esquerdo: X is 2 + 3 dá X = 5. O =:= calcula os dois lados e compara os valores: 5 =:= 2 + 3 dá true.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Em fatorial(N, 120), a regra executa N > 0 e N1 is N - 1 com N ainda sem valor, e a aritmética exige que os valores sejam conhecidos. Por isso dá instantiation_error. O concatena funciona nas duas direções porque usa só unificação, que é simétrica e liga variáveis dos dois lados. A aritmética (is, >) só calcula numa direção, das entradas conhecidas para o resultado; ela não resolve equações.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: a estrutura, com caso base 0 → 1 e passo n × fatorial(n − 1). Diferente: em Haskell a função devolve o valor, enquanto no Prolog o resultado é um argumento extra (F). No Prolog é preciso usar is para calcular e colocar a guarda N > 0. Sem a guarda, o backtracking tentaria a segunda cláusula também com N = 0. Em Haskell, o casamento de padrões já escolhe só a primeira equação que serve.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) vai dar false, e solteiro1(X) vai responder osvaldo, paulo e jake. maximo2(5, 3, M) vai dar duas respostas, M = 5 e M = 3. maximo3(5, 3, 3) vai dar true, o que está errado.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
```

```
?- maximo3(5,3,3).
   maximo3(5,3,3)

?- maximo3(8,3,A).
   maximo3(8,3,8)
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- maximo2(8,3,A).
   maximo2(8,3,8)
   maximo2(8,3,3)

ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16384 Kb, environment variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações. Com 2 cores não existe nenhuma, porque PR, SP e MS fazem fronteira entre si (formam um triângulo) e precisam de 3 cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog escolheu X = ana e Y = ana. Em “tenta ana e ana”, a meta que falhou foi X == Y. Ele voltou ao último ponto de escolha, amigo(Y), e tentou Y = bia. Esse par falhou em gosta, porque bia gosta de chá. Depois tentou Y = caio, e ana e caio gostam de café: primeira resposta. Quando acabaram as opções de Y, ele voltou a amigo(X) e testou bia com todos, sem sucesso. Depois testou caio, e caio com ana deu a segunda resposta.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro2, a meta + casado(X) é executada com X ainda sem valor. Como casado(X) pode ser provado (com X = vern, por exemplo), a negação falha e o resultado é false. Em solteiro1, homem(X) dá um valor a X antes da negação, então ela testa uma pessoa específica. Osvaldo aparece como solteiro porque não existe o fato casado(osvaldo) na base. Pela hipótese do mundo fechado, o que não está na base é falso, e ser pai não implica ser casado no programa.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Na primeira cláusula, a cabeça maximo3(X, Y, X) exige que o terceiro argumento seja igual ao primeiro. Com (5, 3, 3), isso seria 5 = 3, então a unificação falha antes de chegar ao corte. O Prolog passa para a segunda cláusula,

maximo3(, Y, Y), que aceita 3 e responde true. Para corrigir, a unificação deve vir depois do corte:
maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(, Y, Y).
Assim, 5 >= 3 passa, o corte bloqueia a segunda cláusula e M = 5 falha contra 3, dando false.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Em ancestral_ruim, a primeira meta chama a própria regra antes de consumir qualquer fato (recursão à esquerda). Cada chamada gera outra igual, sem avançar, até a pilha acabar. Em ancestral_bom, a regra consome um progenitor antes de chamar a recursão, então cada chamada desce uma geração e a busca termina. As duas são equivalentes em lógica, mas o Prolog executa as metas da esquerda para a direita, e por isso a ordem muda o resultado.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

O programa encontrou 24 colorações com 3 cores, nenhuma com 2 (false) e 1728 com 4. Ele é declarativo porque só descreve o que é uma coloração válida, isto é, as cores possíveis e as regiões vizinhas que precisam ter cores diferentes. Não há laço nem algoritmo de busca escrito no programa: quem faz a busca é o mecanismo de backtracking do Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar é uma função: recebe duas listas e devolve a concatenação, numa só direção. No lógico, concatena é uma relação entre três listas. O mesmo código junta, separa e testa listas, dependendo de quais argumentos estão livres, como em concatena(X, Y, [1, 2, 3]).

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Controle da ordem de resolução: em laco_infinito.pl, ancestral_ruim diz a mesma coisa que ancestral_bom, mas estoura a pilha só porque a chamada recursiva vem primeiro.
Negação como falha e mundo fechado: em controle.pl, solteiro2 deu false só por causa da ordem das metas, e solteiro1 afirmou que osvaldo é solteiro apenas porque casado(osvaldo) não está na base.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Eu escolheria Prolog para problemas baseados em regras e conhecimento: sistemas especialistas, bancos de dados dedutivos, quebra-cabeças e problemas de restrição (como colorir o mapa) e análise de linguagem. Nesses casos, basta descrever o problema, e o backtracking faz a busca. Não escolheria Prolog para sistemas grandes com interface gráfica, muita entrada e saída, cálculo numérico pesado ou desempenho crítico. Nesses casos, Java oferece mais controle sobre a execução, mais bibliotecas e mais eficiência.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
```

```
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

E1 Regras de família: código escrito

```
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.

pares([], []).
pares([H|T], [H|P]) :- H mod 2 =:= 0, pares(T, P).
pares([H|T], P) :- H mod 2 =\= 0, pares(T, P).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).
```

```
potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
    RS \== SC, SC \== PR
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), L), soma(L, Total).
```